Cross-Attention

Contents

1. Overview 2
2. Prerequisites 3
3. What the decoder needs to know 3
4. Input: two sequences instead of one 4
5. Processing: queries from one side, keys and values from the other 5
    5.1 The formula 5
    5.2 By hand 6
    5.3 The same in Keras 8
6. Output: one vector per output word 8
7. What a trained model's cross-attention looks like 10
8. Cross-attention and the attention of the RNN encoder–decoder 11
9. Where cross-attention is used 12
10. Summary 14
11. Sources 14
12. Key terms 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Leads to:** Transformer decoder.
- **Compare with:** Bahdanau (additive) attention; Self-attention (query, key, value).

1. Overview

Key point: Cross-attention (G-507) is attention between two sequences: each word of the sentence being written gets a weighted mix of the words of the input sentence. The computation is the same as **self-attention** (G-1763); only the inputs differ.

In the transformer decoder, the **queries** (G-1607) come from the output sentence being written (French), and the **keys** (G-1011) and **values** (G-2068) come from the encoder's output for the input sentence (English). The weights say how strongly each French position relates to each English word.

Masked self-attention (G-1172) covered the decoder's first attention layer. Its second attention layer is different: in the paper's architecture diagram, two of its three inputs come from the encoder and one from the decoder (Vaswani et al. 2017, Figure 1). The paper calls it **encoder–decoder attention** (G-684); it is now usually called **cross-attention** (SLP3 §13.3).

Cross-attention is a mechanism used in sequence-to-sequence tasks such as translation and summarization. Cross-attention lets a model focus on different parts of the input sequence while it generates the output sequence. This Note compares it with self-attention in three respects (Figure 1):

- what goes in (section 4);
- what happens inside (section 5);
- what comes out (section 6).

The Note then shows the weights of a small trained translation model (section 7), and its link to the attention of the RNN encoder–decoder (section 8).

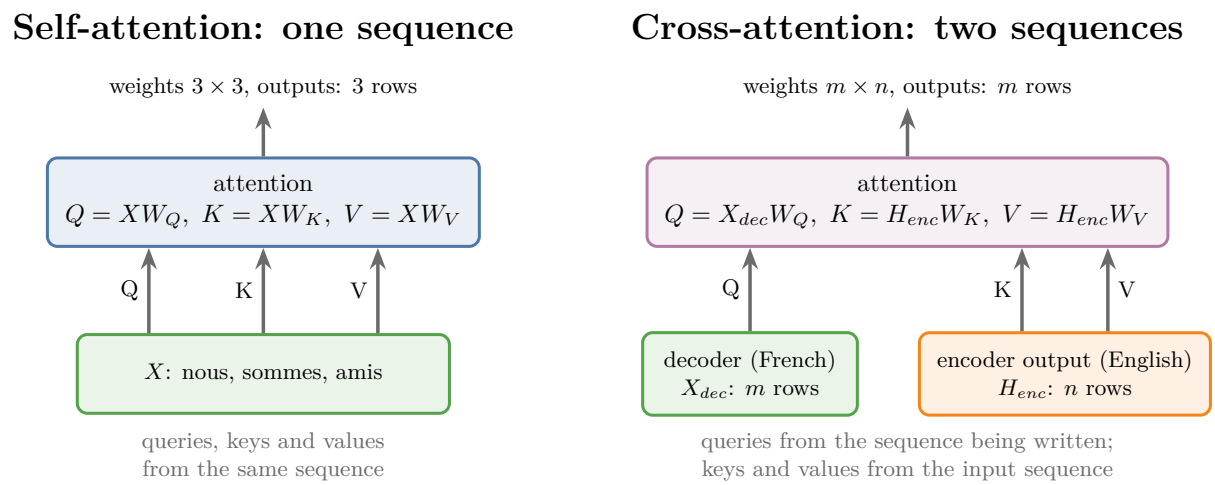


Figure 1: Self-attention takes its queries, keys and values from one sequence. Cross-attention takes its queries from the sequence being written and its keys and values from the encoder's output

2. Prerequisites

- Self-attention step by step: query, key and **value** (G-2068) vectors, W_Q, W_K, W_V.
- **Scaled dot-product attention** (G-1745): $\text{softmax}(QK^T/\sqrt{d_k})\,V$.
- **Multi-head attention** (G-1268): heads, W_O, the parameter count $4d^2 + 4d$.
- Attention mechanism and Bahdanau and Luong attention: attention in the RNN encoder–decoder.
- Transformer encoder: the encoder's output, one vector per input word.
- **Masked self-attention** (G-1172): the decoder's first attention layer.

3. What the decoder needs to know

> **Key point:** The next word depends on two things: the words written so far, and the input sentence. Self-attention handles the first; **cross-attention** (G-507) handles the second.

Take a translation from English into French: "I want ice cream for dessert." becomes "Je veux de la crème glacée pour le dessert." The encoder reads the English sentence and produces one vector per word. The decoder writes the French sentence one word at a time.

Suppose the decoder has written "je veux de la" and must now write the next word. What does the choice depend on?

1. **What it has written so far.** After "je veux de la", only certain words fit. Relating a word to the other words of its own sentence is the job of self-attention; in the decoder it is the **masked self-attention** (G-1172) layer.
2. **What the input sentence says.** "crème" is right because the English sentence says "ice cream". The decoder needs to know which English words matter for the word it is

writing. Losing track of a single input word can reverse the meaning: in "Don't eat the delicious looking pizza", a translation that drops "don't" tells the reader to eat it (StatQuest, "Transformer Neural Networks", 28:30).

The second need is a relationship between two different sequences: for every French word, how strongly it relates to every English word. We can picture it as a table with one row per French word and one column per English word, with "crème" strongly tied to "ice" and "cream", and "veux" to "want".

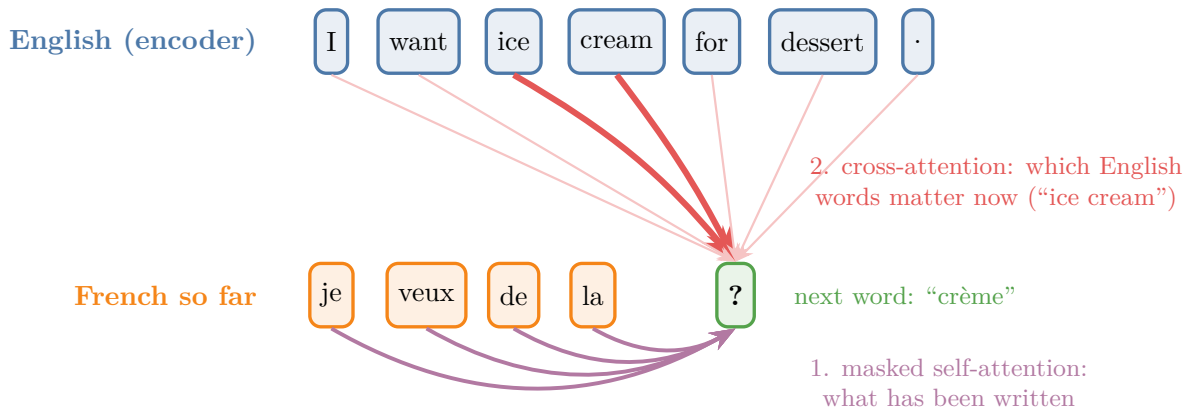


Figure 2: The two things the decoder looks at before writing the next French word. Purple: masked self-attention over the French words written so far. Red: cross-attention over the English sentence, here strongest on "ice" and "cream"

Figure 2 shows both needs for the next word after "je veux de la". The mechanism that computes the French-to-English table is cross-attention:

- **self-attention** (G-1763) relates the items of **one** sequence to each other;
- **cross-attention** relates the items of one sequence to the items of **another**.

4. Input: two sequences instead of one

> **Key point: Self-attention** (G-1763) receives one sequence. Cross-attention receives two: the output sequence and the encoder's output for the input sequence.

Self-attention receives one matrix X, with one row per word: the embeddings of a sentence, or the outputs of the previous layer. For "nous sommes amis" (French for "we are friends"), X has 3 rows.

Cross-attention (G-507) receives two matrices:

- X_{dec}: one row per position of the **output** sentence, coming from the decoder's previous sub-layer;
- H_{enc}: one row per word of the **input** sentence, the final output of the encoder.

A small instance: the English input "we are friends" has $n = 3$ words, so H_{enc} has 3 rows. The French output so far, "nous sommes", has $m = 2$ positions, so X_{dec} has 2 rows. Every output position will score every input word, so the table of scores has this size:

$$m \times n = 2 \times 3$$

In SLP3's notation, where self-attention takes X, "in cross attention the input is the final output of the encoder $H^{enc} = h_1, \ldots, h_n$" (SLP3 §13.3). The input sentence has n words and the output sentence m positions; n and m need not be equal. (The "input" and "output" here are the two sentences, not the input and output of the layer.)

5. Processing: queries from one side, keys and values from the other

> **Key point:** The queries are computed from the output sentence; the **keys** (G-1011) and values from the input sentence. After that, every step is the same as in scaled dot-product attention.

5.1 The formula

> **Key point:** The decoder's vectors are turned into queries, the encoder's output into **keys** (G-1011) and values, and the usual attention formula does the rest: $Q = X_{dec}W_Q$, $K = H_{enc}W_K$, $V = H_{enc}W_V$, then $\text{softmax}(QK^T/\sqrt{d_k})\,V$.

Cross-attention (G-507) has its own three weight matrices W_Q, W_K, W_V, just like **self-attention** (G-1763). The difference is which sequence each one multiplies:

$$Q = X_{dec}\,W_Q$$

$$K = H_{enc}\,W_K$$

$$V = H_{enc}\,W_V$$

$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(SLP3 eq. 13.11 and 13.12). Each French position gets a query vector; each English word gets a key vector and a value vector. The paper describes the same thing: "the **queries** (G-1607) come from the previous decoder layer, and the memory keys and values come from the output of the encoder" (Vaswani et al. 2017, §3.2.3).

From here on nothing is new. Every query is compared with every key by a dot product, the scores are scaled by $\sqrt{d_k}$, a softmax turns each row into **attention weights** (G-225), and each output is the weighted sum of the **value** (G-2068) vectors.

5.2 By hand

> **Key point:** With 4 French positions and 3 English words, the score matrix and the weight matrix are 4×3, and each row of weights sums to 1.

The Notebook takes the pair "we're friends ." and "nous sommes amis", with $n = 3$ English words and $m = 4$ decoder positions (`<start>`, "nous", "sommes", "amis"). Each word has 8 numbers. The numbers are random stand-ins for the encoder output and the decoder states, and W_Q, W_K, W_V are random (untrained); only the shapes and the arithmetic matter here.

	Shape
$Q = X_{dec}W_Q$	4×8
$K = H_{enc}W_K$ and $V = H_{enc}W_V$	3×8 each
Scores $QK^T/\sqrt{8}$ and weights	4×3
Output	4×8

1. **In words:** for the French position "nous", take the dot product of its query with each English **key** (G-1011), divide by $\sqrt{8}$, and turn the three scores into weights with the softmax.
2. **Formula:** for French position i and English word j,

$$w_{ij} = \frac{e^{s_{ij}}}{\sum_{k=1}^{n} e^{s_{ik}}}$$

$$s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

$$y_i = \sum_{j=1}^{n} w_{ij}\, v_j$$

3. **Example:** the scaled scores of "nous" against "we're", "friends" and "." are $0.016,\ 0.505,\ -0.145$ (Notebook). Their exponentials:

$$e^{0.016} = 1.016$$

$$e^{0.505} = 1.657$$

$$e^{-0.145} = 0.865$$

Their sum:

$$1.016 + 1.657 + 0.865 = 3.538$$

Each weight is one exponential divided by the sum:

$$\frac{1.016}{3.538} = 0.287$$

$$\frac{1.657}{3.538} = 0.468$$

$$\frac{0.865}{3.538} = 0.244$$

The output for "nous" is a mix of English **value** (G-2068) vectors:

$$0.287\, v_{\text{we're}} + 0.468\, v_{\text{friends}}$$

$$+0.244\, v_{.}$$

The full weight matrix (rows: French positions; columns: English words):

French position	we're	friends	.
`<start>`	0.472	0.237	0.291
nous	0.287	0.468	0.244
sommes	0.213	0.603	0.184
amis	0.423	0.349	0.228

Every row sums to 1 (up to rounding in the last digit).

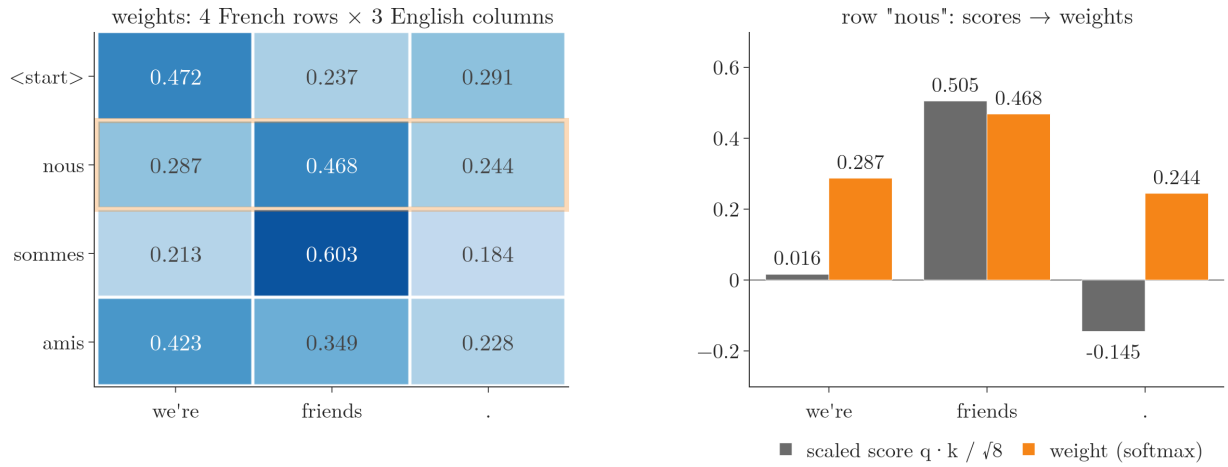


Figure 3: Left: the hand-computed weight matrix, one row per French position and one column per English word; the orange frame is the row of "nous". Right: that row built from its three scaled scores by the softmax

In Figure 3, the matrix has 4 rows and 3 columns because the **queries** (G-1607) come from the 4 French positions and the keys from the 3 English words. On the right, the softmax keeps the order of the scores: "friends" has the largest score (0.505) and gets the largest weight (0.468).

5.3 The same in Keras

> **Key point:** Keras' `MultiHeadAttention` does cross-attention when the query and the **value** (G-2068) are different sequences.

Keras' `MultiHeadAttention` layer takes a `query`, a `value` and an optional `key` (the **key** (G-1011) defaults to the value) (Keras documentation). Passing the decoder states as the query and the encoder output as the key and value gives cross-attention. With one head, the same weights and the output matrix set to the identity, the layer returns the hand-computed weights and outputs; the largest difference is 9×10^{-8} (Notebook).

Python: Cross-attention (G-507) by hand and with Keras.

```
Q = X_dec @ Wq                          # queries from the French side   (4
↪  x 8)
K = H_enc @ Wk                          # keys from the English side     (3
↪  x 8)
V = H_enc @ Wv                          # values from the English side   (3
↪  x 8)
A = softmax_rows(Q @ K.T / np.sqrt(8))
Y = A @ V

out, w = mha(query=X_dec, value=H_enc, key=H_enc,
↪  return_attention_scores=True)
```

The same layer, given `X_dec` as both query and value, does **self-attention** (G-1763) and returns a 4×4 weight matrix. The layer is identical; only its inputs decide which kind of attention it is. A cross-attention layer therefore also has the same parameters as a self-attention layer: at the paper's sizes ($d_{model} = 512$, 8 heads), 1,050,624 in both cases, the $4d^2 + 4d$ of the **multi-head attention** (G-1268) (Notebook).

6. Output: one vector per output word

> **Key point: Cross-attention** (G-507) returns as many vectors as the output sequence has positions, whatever the length of the input. Each is a weighted mix of the input words.

Self-attention (G-1763) on "nous sommes amis" returns 3 contextual embeddings, one per word. Each is a mix of the value vectors of the same 3 words: "nous" might be built 80% from "nous" and 10% each from "sommes" and "amis".

Cross-attention returns one vector per row of the query, that is, per output position. Each is a mix of the **value** (G-2068) vectors of the **input** words: the output for "amis" might be 30% "we're", 60% "friends" and 10% ".". The number of outputs follows the output sentence, never the input. In the Notebook, a 7-word English input gives a 4×7 weight matrix and still 4 outputs of 8 numbers.

	Self-attention	Cross-attention
Input	one sequence X (m rows)	output sequence X_{dec} (m rows) and encoder output H_{enc} (n rows)
Queries from	X	X_{dec}
Keys and values from	X	H_{enc}
Weight matrix	$m \times m$	$m \times n$
Output	m vectors, each a mix of the same sequence	m vectors, each a mix of the input sequence

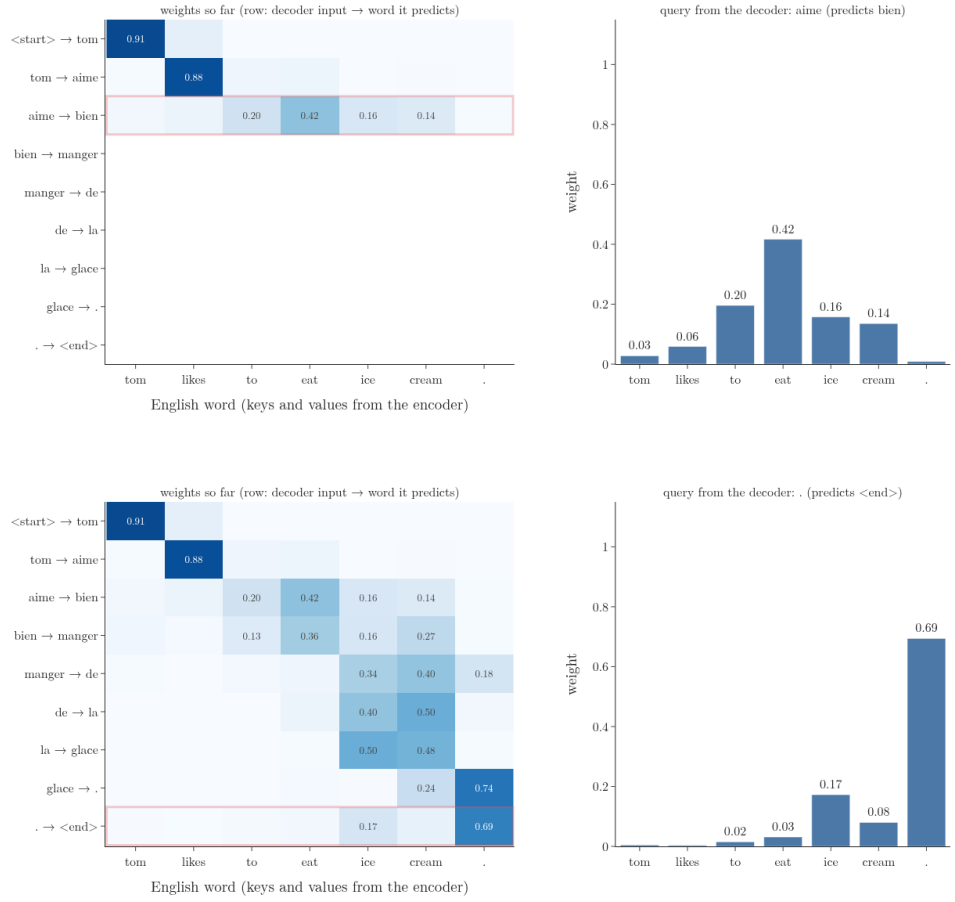


Figure 4: Cross-attention of the trained model of section 7 on "Tom likes to eat ice cream.", one French position per frame. Left: the 9×7 weight grid filling row by row. Right: the current query's weights over the English words, whose keys and values come from the encoder

Figure 4 shows this with a trained model: each French position sends one query to the encoder's output and gets back its own mix of the 7 English words. The grid has 9 rows and 7 columns, output length by input length, and section 7 reads what the weights mean.

7. What a trained model's cross-attention looks like

> **Key point:** In a small English-to-French transformer trained in the Notebook, each French word puts most of its **cross-attention** (G-507) weight on the English word or words it translates.

The weights in section 5 were random. To see what training produces, the Notebook trains a small English-to-French transformer on the English–French sentence pairs of the Keras examples (from the Tatoeba project): about 131,000 pairs of up to 10 words each. The model has one encoder block and one decoder block, $d_{model} = 128$ and 4 heads; the decoder block is explained in the transformer decoder. After 6 epochs (about 4 minutes on a GPU) it predicts the correct next French word 78.4% of the time on 5,000 pairs it never saw, when given the correct previous words.

The two English sentences of Figure 5 were removed from the training data. The decoder is given the reference French translation, as in training, and the figure shows the cross-attention weights, averaged over the 4 heads.

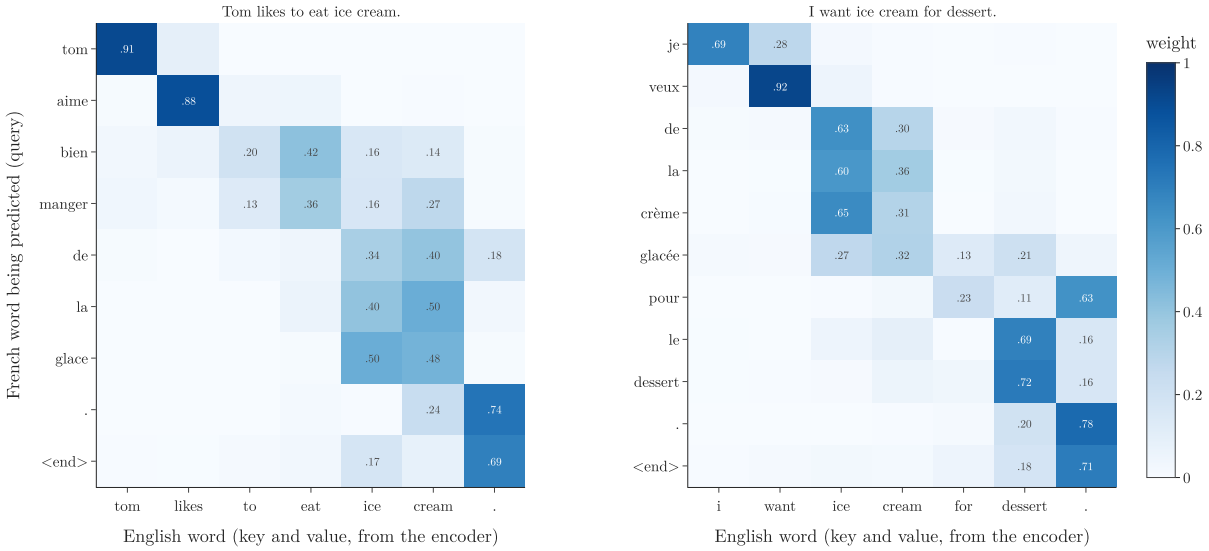


Figure 5: Cross-attention weights of the trained model on two sentence pairs it never saw. Rows: the French word being predicted; columns: the English words. Weights below 0.1 are not written

Most rows put their largest weight on the English word they translate:

French word predicted	Largest weight on
tom	tom (0.91)
aime	likes (0.88)
glace	ice (0.50) and cream (0.48)
je	i (0.69)

French word predicted	Largest weight on
veux	want (0.92)
crème	ice (0.65)
dessert	dessert (0.72)

Nobody told the model which English word matches which French word; it was trained only to predict the next French word. The weights follow the word order where the languages differ, too: "crème glacée" looks at "ice cream", and the two French words before it ("de la"), which have no English counterpart, already look at "ice" and "cream". Not every row is that clean. The row predicting "pour" puts 0.63 on the full stop and only 0.23 on "for", and the rows for the final full stop and `<end>` look at the English full stop, as expected. A single picture of **attention weights** (G-225) shows where a layer looks, not why (the heads of a trained model).

8. Cross-attention and the attention of the RNN encoder–decoder

> **Key point:** Cross-attention does what Bahdanau and Luong attention did in the RNN encoder–decoder: at each output step, weigh every input position and mix them. The transformer simply computes it with queries, **keys** (G-1011) and values.

Cross-attention (G-507) may look familiar. In the attention mechanism, the RNN decoder received a fresh context vector at every step:

$$c_i = \sum_j \alpha_{ij}\, h_j$$

where h_j are the encoder's hidden states and α_{ij} measures how relevant input word j is to output step i. Bahdanau attention computes the score with a small neural network, Luong attention with a dot product (Bahdanau versus Luong attention). In both, the decoder side asks and the encoder side answers, which is exactly the query–key–**value** (G-2068) split of cross-attention: the decoder state plays the query, the encoder states play the keys and the values (why self-attention is attention).

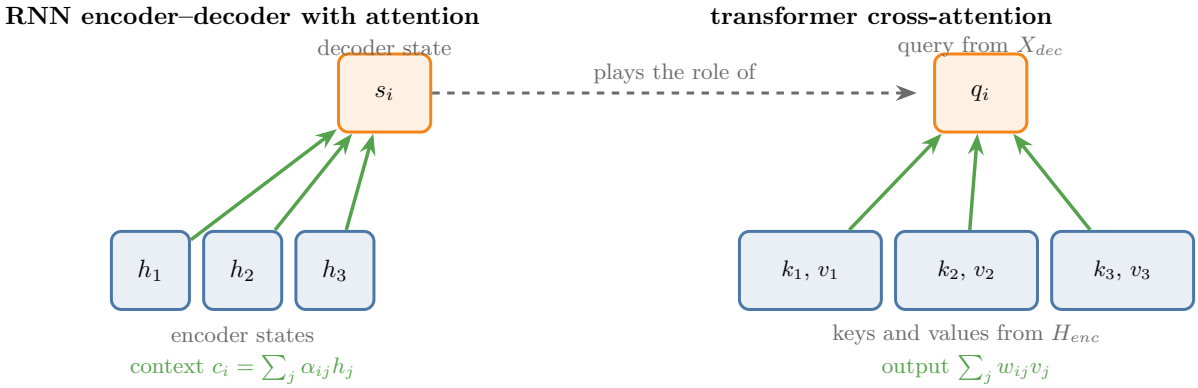


Figure 6: Left: in the RNN encoder–decoder, the decoder state s_i weighs the encoder states h_j to build its context vector. Right: in cross-attention, the query q_i from the decoder weighs the keys from the encoder and mixes the values

Figure 6 puts the two side by side: the decoder side asks, the encoder side answers, in both designs.

The paper says so directly: **encoder–decoder attention** (G-684) "allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models", citing Bahdanau et al. among others (Vaswani et al. 2017, §3.2.3). What changed is the machinery around it: learned W_Q, W_K, W_V, scaling, several heads, and no RNN, so every output position is computed at once during training.

9. Where cross-attention is used

> **Key point:** Wherever one sequence is generated from another, including sequences of different kinds: text from text, text from sound, images from text.

Cross-attention (G-507) is used wherever a model produces one sequence while looking at another:

- **Machine translation:** the decoder attends to the source sentence (SLP3 §13.3).
- **Speech recognition:** Whisper is an encoder–decoder transformer whose decoder writes text while cross-attending to the encoded audio (Radford et al. 2022, §2.2 and Figure 1).
- **Text-to-speech:** the Transformer TTS model of Li et al. (2019) generates speech features with a transformer decoder that attends to the encoded text.
- **Text-to-image generation:** latent diffusion models, the basis of Stable Diffusion, condition the image on the text through cross-attention layers (Rombach et al. 2022, §3.3).
- **Image captioning:** "Show, Attend and Tell" lets an RNN decoder attend to regions of the image while it writes the caption, the same idea before transformers (Xu et

al. 2015).

Tasks whose input and output are of different kinds, such as audio and text, or text and images, are called **multimodal** (G-1274); cross-attention is the standard way to connect the two sides.

10. Summary

	Self-attention	Cross-attention
Relates	the items of one sequence	the items of two sequences
Queries	from the sequence itself	from the output sequence (decoder)
Keys and values	from the sequence itself	from the input sequence (encoder output)
Weights	$m \times m$	$m \times n$
Outputs	m	m, one per output position
Parameters	$4d^2 + 4d$	the same

- The decoder's next word depends on what it has written (**masked self-attention** (G-1172)) and on the input sentence (cross-attention), so the decoder needs both layers: losing one input word, such as "don't", can reverse the meaning.
- **Cross-attention** (G-507): $Q = X_{dec}W_Q$, $K = H_{enc}W_K$, $V = H_{enc}W_V$, then $\text{softmax}(QK^T/\sqrt{d_k})\,V$. The **queries** (G-1607) come from the output side because each output position is the one asking which input words it needs.
- The weight matrix is $m \times n$ and there are m outputs, so the output follows the length of the sentence being written, whatever the input length.
- The same Keras layer does **self-attention** (G-1763) or cross-attention, depending only on whether query and value are the same sequence, so the two cost the same parameters ($4d^2 + 4d$).
- In a small trained translation model, most French words put their largest cross-attention weight on the English words they translate, although it was trained only to predict the next French word; so the alignment between the two sentences is learned, not given.
- Cross-attention is the transformer form of the **encoder–decoder attention** (G-684) of Bahdanau and Luong, and it connects the two sides of translation, speech recognition, text-to-speech and text-to-image models, because each of them writes one sequence while looking at another.

11. Sources

Built from

- CampusX, "Cross Attention in Transformers | 100 Days Of Deep Learning | CampusX", YouTube, https://www.youtube.com/watch?v=smOnJtCevoU
- StatQuest with Josh Starmer, "Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=zxQyTK8quyY. 28:30–29:00 (encoder–decoder attention keeps track of the significant input words; the "don't eat the delicious looking pizza" example).

Other references

- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Figure 1; section 3.2.3 (encoder–decoder attention: queries from the decoder, keys

and values from the encoder output; "mimics the typical encoder-decoder attention mechanisms").
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026). Chapter 13, section 13.3 (cross-attention, also called encoder–decoder attention or source attention; eq. 13.11 and 13.12).
- Keras API documentation: `MultiHeadAttention` layer (`query`, `value`, `key` arguments), keras.io/api/layers/attention_layers/multi_head_attention.
- Radford, A. et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. arXiv:2212.04356. Section 2.2 (encoder–decoder transformer) and Figure 1 (cross attention).
- Li, N., Liu, S., Liu, Y., Zhao, S. and Liu, M. (2019). Neural Speech Synthesis with Transformer Network. *AAAI 2019*. arXiv:1809.08895.
- Rombach, R. et al. (2022). High-Resolution Image Synthesis with Latent Diffusion Models. *CVPR 2022*. arXiv:2112.10752. Section 1 and section 3.3 (conditioning through cross-attention).
- Xu, K. et al. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. *ICML 2015*. arXiv:1502.03044.
- Tatoeba sentence pairs, distributed as the English–French dataset of the Keras examples (fra-eng.zip; manythings.org/anki).

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Cross-attention (G-507)	Attention between the two halves of a transformer: the queries come from the decoder and the keys and values from the encoder's output, so each output word can use the input sentence.
Encoder–decoder attention (G-684)	The paper's name for cross-attention, also called source attention: the decoder's attention layer whose queries come from the output sentence and whose keys and values come from the encoder, so each word being written can draw on the input sentence.
X_{dec} (G-44)	The decoder's numbers for the output sentence (its representation), one row per output position, which go into cross-attention.

Term	Meaning
Self-attention (intra-attention) (G-1763)	Attention in which the words of one sequence attend to each other (the queries, keys and values all come from that sequence); it gives each word a new vector that depends on the words around it.
Query, key, value vectors (G-1607)	Three vectors made from each word's embedding by learned matrices: a word's query is compared with every word's key by dot products to get attention weights, and those weights mix the value vectors into the word's new vector.
Key (G-1011)	In attention, the vector each word offers to be compared with a query; the dot product of the query with a key scores how much attention that word gets.
Value (G-2068)	In self-attention, the vector each word contributes to the weighted sum that becomes a word's new vector; it comes from the word's embedding through a learned value matrix.
Masked self-attention (G-1172)	Self-attention in which each word can take information only from itself and earlier words: the later words' scores get $-\infty$ before the softmax, so their weights are 0, $\mathrm{softmax}(QK^T/\sqrt{d_k} + M)\,V$. It lets the decoder train on whole target sentences in one pass without seeing the future.
Scaled dot-product attention (G-1745)	Self-attention in which every query–key dot-product score is divided by $\sqrt{d_k}$ before the softmax, so the scores do not spread so wide that the softmax saturates: $\mathrm{softmax}(QK^T/\sqrt{d_k})\,V$.
Multi-head attention (G-1268)	Several self-attentions (heads) run in parallel on the same input, each with its own weights so each can capture a different kind of relation between words; their outputs are joined and mixed by one more learned matrix, W_O.
Attention weight (G-225)	How much one word draws on another word when building its new vector: an attention score after the softmax, so the weights of one word are non-negative and sum to 1.

Term	Meaning
Multimodal model (G-1274)	A model that takes in or produces more than one modality.
H_{enc} (G-30)	The output of the last encoder block: one vector per input word, read by every decoder block.
Key and value (G-1010)	In attention, the vectors of the words being looked at: keys are compared with the query to get the weights, and values are mixed with those weights; in cross-attention both come from the encoder's input words.
Alignment (G-191)	Which input words each output word relates to; read from the cross-attention weights.
Multimodal (G-1275)	Having more than one mode, that is more than one peak (two peaks: bimodal), such as a robot's belief peaking at three doors; it shows the data point to several separate values, so one mean or one best guess would hide them.